

Task & Objective Management

AI-assisted task planning from an uploaded project document

Area	Choice
Language	JavaScript (ES modules on the client, CommonJS on the server)
Frontend	React 19 + Vite 8, React Router 7, hand-written CSS
Backend	Node.js + Express 5
Database	MongoDB Atlas (cloud)
ORM / ODM	Mongoose 9
External API	Google Gemini (generativelanguage REST API), free tier
Deployment target	Vercel

This document describes what the module does, how every layer is built, why each decision was made, what was tested and what is deliberately not included. It is written to be read alongside the code.

1. What this module is

The wider product is an employee management system split across several modules. This document covers **Module 3 only: Task & Objective Management**. Other modules (authentication, HR records, payroll) mount into the same Express app and the same React router, but none of their code is part of this work.

1.1 The problem it solves

A manager receives a project brief - a client document, a specification, a set of meeting notes. Turning that document into assigned, estimated, tracked tasks is manual, slow, and the thing everybody puts off. The module does that first pass automatically.

1.2 What a user can actually do

- **Upload a document** (PDF, Word, Markdown, CSV or plain text) or paste text, and get back a draft set of tasks with titles, descriptions, estimates, priorities, a department and, where possible, a named owner.
- **Review and edit that draft** before anything is saved - change the title, department, owner, priority or hours, or drop a task entirely.
- **Assign work to a person or to a whole department** (a department is the "room" in the requirement - a task can sit against a department with no individual owner).
- **Track progress** through a checklist of subtasks; the completion percentage is calculated from it, never typed in.
- **Ask questions on a task** - a question stays marked open until somebody replies to it.
- **Attach files** to a task and download them again.
- **See the whole department at a glance** on the orbit view, where distance from the centre is priority and planet size is estimated effort.
- **Group work into projects** and watch one roll up: a project cuts across departments, and its progress is counted from every task under it, weighted by estimate.
- **See when work actually moved** — a task records when it was created, assigned, started and finished, not just when the record was made.

The single most important design decision

Nothing the AI proposes is saved automatically. The document is read, tasks are drafted, and then the manager edits and approves them. A tool that silently creates a dozen tasks from a half-understood brief is worse than no tool at all - it puts work in the system that nobody agreed to and everybody has to clean up.

2. Requirement compliance

Each rule from the course guidelines, and where it is met.

Requirement	Status	Where / how
Language: JavaScript	Met	All source is JavaScript. No TypeScript.
Frontend: React.js	Met	React 19 with Vite as the build tool.
Backend: Node.js, Express.js	Met	Express 5 on Node 24.
Database: MongoDB	Met	MongoDB Atlas, six collections.
ORM: Mongoose	Met	Mongoose 9; every collection has a schema in backend/model/.

Requirement	Status	Where / how
Deployment: Vercel	Ready	No filesystem writes anywhere; uploads are held in memory and stored in MongoDB, so the read-only serverless filesystem is never touched. The Mongoose connection promise is cached for warm container reuse.
External API integration	Met	Google Gemini, called over its REST API. Free tier is sufficient.
Responsive / mobile-friendly	Met	Media queries in 7 of 9 stylesheets. Verified in-browser: no horizontal page overflow at mobile width; the wide task table scrolls inside its own container rather than pushing the page sideways.
No drag-and-drop builder / CMS	Met	Hand-written JSX and CSS. No Elementor, Wix, WordPress, Bootstrap builder or page builder of any kind.
Not core PHP / Firebase / Supabase / Django	Met	None used.
Version control with individual commits	Not yet done	The project folder is not currently a git repository. See section 12.

Action needed before submission

The guidelines state that copy-pasted code without commit history will be penalised. This folder has a .gitignore but is not a git repository yet. Initialise one, and have each team member commit their own work under their own name so the history shows individual contribution.

3. How the pieces fit together

Three layers, one direction of dependency. The browser never talks to MongoDB, and the database layer never knows what HTTP is.



3.1 Why the layers are separated this way

- **Controllers never build a database query.** They read the request, validate it, call one service function and choose a status code. That means the rules about what a task *is* live in one file, not scattered through request handlers.
- **Services never touch req or res.** They can be called from a script, a test or a different transport without change - the seed script uses the models directly for exactly this reason.
- **The frontend has exactly one file that calls fetch.** Base URL, error shape and the multipart/JSON decision live in lib/api.js. No page constructs a URL itself.

3.2 Development vs production URLs

The browser always calls `/api/...` on its own origin. In development Vite proxies that to Express on port 4000 (see `vite.config.js`). In production both are served from the same origin. The result is that **no CORS configuration and no environment switch is needed** - the same relative path is correct everywhere.

```
// frontend/vite.config.js
server: {
  proxy: { '/api': { target: 'http://localhost:4000', changeOrigin: true } }
}
```

4. The database

MongoDB Atlas, six collections, all defined with Mongoose schemas. Relationships use `ObjectId` references with `ref`, resolved on read with `.populate()`.

4.1 Collections at a glance

Collection	Holds	Key references
departments	The four teams. Name, a short mark used as an icon, a one-line blurb.	none
employees	People who can own a task. Name, email, job title, department, skills, avatar colour, isActive.	none (department is stored by name)
objectives	A project . Title, client, description, start and target dates, status, and where it was imported from. Deliberately cuts across departments.	none
tasks	The core record. Title, description, department, priority, status, estimate, spent, due date, embedded subtasks, attachment ids, source.	assignee -> employees, objective -> objectives, attachments -> attachments
comments	One thread per task. A 'question' stays open until replied to.	task -> tasks, author -> employees, replyTo -> comments
attachments	Uploaded files, stored as binary inside the document.	task -> tasks

4.2 The task schema in full

```
const taskSchema = new mongoose.Schema({
  title:      { type: String, required: true, trim: true },
  description: { type: String, default: '', trim: true },

  department: { type: String, required: true, trim: true, index: true },
  assignee:   { type: ObjectId, ref: 'Employee', default: null, index: true },
  objective:  { type: ObjectId, ref: 'Objective', default: null },

  priority:   { type: String, enum: ['critical', 'high', 'medium', 'low'],
               default: 'medium', index: true },
  status:     { type: String, enum: ['todo', 'in_progress', 'review', 'done'],
               default: 'todo', index: true },

  estimateHours: { type: Number, default: 4, min: 0 },
  spentHours:    { type: Number, default: 0, min: 0 },
  dueDate:       { type: Date },

  subtasks:     [subtaskSchema], // embedded, each with its own _id
  attachments:  [{ type: ObjectId, ref: 'Attachment' }],

  source:       { type: String, enum: ['manual', 'ai'], default: 'manual' },
  aiReason:     { type: String, default: '' },
}, { timestamps: true })
```

Why subtasks are embedded but attachments are referenced

Subtasks are small, always loaded with their task, never queried on their own, and bounded in number - so embedding them means one read instead of two and keeps the checklist atomic with the task. Attachments are the opposite: they carry file bytes. Embedding those would drag megabytes into every task list query, so they live in their own collection and the task holds only their ids.

Why file bytes are in MongoDB rather than on disk

Vercel's filesystem is read-only. Writing uploads to an `uploads/` folder works perfectly on a laptop and then fails in production - the classic deployment trap. Storing the bytes in the document sidesteps it entirely. The upload middleware caps a file at 5 MB, well under MongoDB's 16 MB document limit, and the `data` field is declared `select: false` so listing attachments never loads the bytes.

```
data: { type: Buffer, required: true, select: false }
```

4.3 Projects, and why progress is never stored

An **objective** is a project: the thing a group of tasks is for. It is the one record that cuts across departments, which is exactly why it exists alongside them — a supplier portal needs engineering, design and a launch campaign, and the department view can only ever show one of those at a time.

```
{
  title, description, client,
  startDate, dueDate,
  status:      'planning' | 'active' | 'paused' | 'delivered',
  source:      'manual' | 'ai',      // did it come from a document?
  sourceDocument: 'courier_brief.pdf', // which one
  summary:      '...'              // what the reader understood
}
```

Notice what is *not* in there: no progress, no task count, no percentage. Every number on the project page is counted from the tasks pointing at it, at the moment it is asked for. A stored percentage and the work underneath it disagree the moment anybody ticks anything.

How project progress is worked out

Weighted by estimated hours, not by counting tasks. Counting tasks would let five one-hour chores outweigh a forty-hour build, so a project could read 80% complete with all the real work still ahead of it. Hours are the closest thing to effort the system records, so they are what the percentage is built from. With no estimates at all it falls back to a plain average rather than dividing by zero.

```
progress = sum(task.progress x task.estimateHours) / sum(task.estimateHours)
```

Project health is derived too

Health	When
Delivered	Every task in it is done.
Late	Any task is overdue, or the project's own target date has passed with work still open.
Needs owners	Nothing late, but open tasks have nobody assigned.
On track	None of the above.
No tasks yet	The project exists but nothing points at it.

Deleting a project does not delete the work

The tasks are real and somebody may still be doing them. Deleting the project releases them back to their departments with their objective set to null — they appear under "Not part of a project" rather than vanishing. Losing a dozen real tasks because somebody tidied up a project record would be unforgivable.

4.4 The dates a task records

Four timestamps, all written by the service layer as the task moves rather than typed in by anyone, so they cannot be edited into saying something that did not happen:

Field	Set when	Cleared when
<code>createdAt</code>	The record is made (Mongoose timestamps).	never

Field	Set when	Cleared when
assignedAt	Somebody is first assigned.	The assignee is removed.
startedAt	Status leaves 'to do'.	The task returns to 'to do'.
completedAt	Status becomes 'done'.	The task is reopened.
updatedAt	Any change (Mongoose timestamps).	never

Checklist items carry their own `completedAt` as well, so the task page can show when each step was ticked. Clearing on reverse is the part worth pointing at: a reopened task with a completion date on it is a lie, and reports built on it would inherit the lie.

4.5 Fields that are calculated, never stored

Progress, days remaining, overdue state and remaining hours are all worked out when a task is read, in `service/task_service.js` -> `decorate()`. None of them is a column.

Field	How it is derived
progress	100 if status is done. Otherwise, if the task has subtasks, the percentage of them ticked. Otherwise a value implied by the status (todo 0, in_progress 40, review 80).
daysLeft	Whole days between now and <code>dueDate</code> ; null when there is no due date.
overdue	Not done, has a due date, and <code>daysLeft</code> is negative.
remainingHours	<code>estimateHours</code> scaled by the portion still outstanding, rounded to one decimal.
subtasksDone	Count of ticked subtasks.
id	String form of <code>_id</code> , added so the client never handles a raw <code>ObjectId</code> .

A question the teacher is likely to ask: why not just store a percentage?

Because a stored number and a checklist will disagree the moment somebody ticks a box. Two sources of truth for one fact is a bug waiting to happen. Deriving it on read means the orbit view, the task list and the task page cannot show three different numbers.

4.6 Status moves by itself, but only forwards

Two rules live in the service layer, applied whenever a subtask is toggled or a task updated:

- Ticking the first subtask on a task that is still *to do* moves it to *in progress*. Work has visibly started, so the board should say so.
- Ticking the last outstanding subtask moves the task to *in review* - not to *done*. Finishing the checklist means the work is ready to be checked, and a person still decides when it is actually complete.

4.7 Indexes

Indexes are declared on `department`, `assignee`, `priority` and `status` on tasks, on task for comments and attachments, and on `department` for employees. These are exactly the fields the board query filters and sorts by.

4.8 Seeding

`npm run seed` clears the six collections this module owns and reloads them from `backend/data/dummy_data.js`: 4 departments, 12 employees, 5 objectives, 27 tasks and 3 comments. Employee skills are inferred from job title, so there is no second data file to keep in step. It is safe to re-run.

5. The backend

5.1 Every endpoint

Method & path	Does
GET /api/tasks/board?department=&include Done=	Everything the first page needs in one request: all departments with their counts, plus the tasks for the selected one.
GET /api/tasks/options	Fills the dropdowns on the new-task form: departments, employees, objectives, and the valid priority and status values.
GET /api/tasks/:id	One task, plus the employee list and the comment thread.
POST /api/tasks	Create a task.
PATCH /api/tasks/:id	Update any editable field.
DELETE /api/tasks/:id	Delete a task, and its comments and attachments with it.
POST /api/tasks/:id/subtasks	Add a checklist item.
PATCH /api/tasks/:id/subtasks/:subtaskId	Tick or untick a checklist item.
POST /api/tasks/:id/comments	Post a comment or a question. Replying resolves the question.
POST /api/tasks/:id/attachments	Upload a file (max 5 MB).
GET /api/tasks/:id/attachments/:fileId	Download a file.
DELETE /api/tasks/:id/attachments/:fileId	Remove a file.
GET /api/objectives	Every project with its rolled-up progress, health, department spread and people.
GET /api/objectives/:id	One project, its rollup, and every task filed under it.
POST /api/objectives	Create a project.
PATCH /api/objectives/:id	Rename, retarget or change its status.
DELETE /api/objectives/:id	Delete the project. Its tasks survive, released back to their departments.
GET /api/ai/status	Reports whether a Gemini key is configured, which model, and the supported file types.
POST /api/ai/analyse	Read a document or pasted text, return a draft plan. Saves nothing.
POST /api/ai/create-tasks	Create the tasks the manager kept, and file them under a project so the document survives as one thing.

A subtle thing worth being able to explain

In `task_routes.js`, `GET /board` and `GET /options` are declared *before* `GET /:id`. Express matches routes in order, so if they came after, a request for `/board` would be captured by `/:id` and the server would try to look up a task whose id is the string "board".

5.2 Errors are shaped in exactly one place

Handlers do not write error responses. They throw, or pass to `next`, and one error handler at the bottom of `server.js` decides what the browser sees. Mongoose validation failures become 400s with the field message; bad ObjectIds become a readable 400; anything unexpected is logged in full on the server and reported generically, so an internal message never leaks to a user.

```
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err)

  if (err.name === 'ValidationError') {
    const message = Object.values(err.errors).map(e => e.message).join(' ')
    return res.status(400).json({ error: message || 'Some fields are missing or invalid.' })
  }
})
```



```
    }  
    if (err.name === 'CastError') {  
      return res.status(400).json({ error: 'That record id is not valid.' })  
    }  
  
    console.error('[api]', err)  
    res.status(500).json({ error: 'Something went wrong on our side.' })  
  })
```

Controllers wrap async handlers in a small helper so a rejected promise reaches that handler instead of hanging the request:

```
const asyncRoute = (handler) => (req, res, next) =>  
  Promise.resolve(handler(req, res, next)).catch(next)
```

5.3 Uploads

Multer with memory storage, used twice with different limits: 5 MB for task attachments, 8 MB for imported documents. Nothing is written to disk. Multer's own errors are translated into the same JSON shape as the rest of the API, so the client has one error format to handle.

Downloads are sent with `Content-Disposition: attachment`. That is deliberate: it means an uploaded HTML or SVG file can never be rendered as a page inside the application's own origin, which would otherwise be a stored cross-site scripting hole.

6. The AI feature, step by step

This is the module's headline feature and the part most likely to be questioned in detail.

6.1 The flow

1. Manager drops a file (or pastes text) and optionally adds notes.
2. `POST /api/ai/analyse` (multipart)
3. `document_parser.js` -- extracts plain text from PDF / DOCX / TXT / MD / CSV
4. Departments and employees are loaded from MongoDB
5. Is GEMINI_API_KEY set?
 - yes -> `gemini.js` : one REST call, JSON-schema-constrained reply
 - on failure -> fall through to the built-in reader
 - no -> `fallback_planner.js` : structural parse of the document
6. The result is normalised into one fixed shape the review screen expects
7. Response returns to the browser. NOTHING HAS BEEN SAVED.
8. Manager edits, unticks, adjusts.
9. `POST /api/ai/create-tasks` -> a project is created from the document, then `insertMany` with every task pointing at it

6.2 Reading the document

`service/document_parser.js` handles five formats, entirely in memory:

Format	Library	Note
.pdf	pdf-parse v2	v2 exports a <code>PDFParse</code> class, not the v1 function. It also marks page boundaries like "-- 1 of 3 --", which is useful in a viewer and noise here, so those markers are stripped.
.docx	mammoth	<code>extractRawText</code> - formatting is discarded, only the words matter.
.txt / .md / .csv	none	Read straight from the buffer as UTF-8.

Text is then normalised (line endings, runs of blank lines) and capped at 20,000 characters. If the file produced no text at all - a scanned image, for instance - the user is told plainly that it needs OCR first, rather than being handed an empty result.

6.3 Gemini

Called over its REST API with the built-in `fetch`, not an SDK, so there is one fewer dependency to keep in step. Model: `gemini-2.5-flash` (configurable). Temperature 0.3, because this is an extraction task, not a creative one. A 45-second timeout is enforced with an `AbortController`.

The important detail is that the reply is **schema-constrained**. Gemini is given an OpenAPI-style `responseSchema` and `responseMimeType: 'application/json'`, so it can only answer with JSON matching that shape. There is no prose to parse out of a chat reply and no fragile regex - the response is either valid JSON of the right shape or an error.

```
generationConfig: {
  temperature: 0.3,
  responseMimeType: 'application/json',
  responseSchema: PLAN_SCHEMA // summary + tasks[] with typed fields
}
```

The prompt supplies the department list and the full team roster with job titles and skills, and instructs the model to only suggest an owner whose skills genuinely fit, and never to invent facts that are not in the document.

6.4 The built-in reader (the fallback)

Used when no key is configured, or when the Gemini call fails for any reason. It returns the *same shape*, so nothing downstream can tell the difference - and the response always names which engine produced it, so the

interface can be honest about it.

It works structurally rather than semantically:

- **Headings** are detected from Markdown hashes, a trailing colon, all-caps, or - added during this work - the shape of a short capitalised line with no terminal punctuation, because a PDF keeps none of Markdown's marks and a Word heading carries its weight in the style, not the text.
- **Bullets** are matched loosely. A bullet character survives a PDF badly: depending on the font a dot can come back as a quote, a middle dot, or a private-use character from Word's Symbol font, so the pattern accepts any single leading symbol followed by a space.
- **Plain prose** only becomes a task when it reads like an instruction - it must start with an action verb.
- **Priority** comes from urgency words; **estimate** from line length and the kind of verb used.
- **Department** is chosen by matching words in the line to the skill tags on employee records, then picking the department whose people carry those skills most often.

Known and deliberate limitation

The built-in reader looks at each line on its own. It therefore cannot connect a sentence in a Constraints section - "patient registration and the booking API are critical" - back to the bullets further up the page, so everything comes out as medium priority. Gemini reads the whole document at once and gets this right. This is a genuine difference in capability, it is stated in the interface, and it is the single best reason to set the free API key before a demo.

7. The frontend

7.1 Pages

Route	Page	What it does
/	task_orbit	A department's work as a solar system, then the projects that department is carrying, then any single tasks that belong to no project.
/tasks/new	new_task	Two modes on one page: type a single task, or hand over a document. A separate page for each would be two places to look for "add work".
/tasks/:id	task_detail	One task: checklist, files, question thread, its dates, its project, and the controls to change status, priority and owner.
/projects	projects	Every project with a progress bar, health and target date. The view the orbit cannot give, because a project cuts across departments.
/projects/:id	project_detail	One project end to end: overall progress, every task <i>and its checklist items</i> with tick state and completion times, the stage and department breakdown, who is on it, and the dates.

7.2 The orbit view

Tasks are drawn as planets. The rules are kept deliberately simple so they can be explained in one breath, and the legend on the page states them:

- Ring position and colour = **priority**, and never anything else. Critical is the innermost ring.
- Planet size = **estimated hours**, clamped so a 40-hour task reads as bigger than a 2-hour one without swallowing its neighbours.
- The white arc = **progress**.
- A red halo - a gap in the background colour, then a solid ring - = **past its due date**. It is the only marking on a planet that is not priority, size or progress, which is what makes an overdue task findable at a glance without reading a single label. It is driven by the same `isOverdue` rule as the counts, so the halo, the sidebar and the department chip can never disagree.

Each ring has a cap on how many planets it will show, and the ones kept are those with the soonest deadline - so anything trimmed is the least pressing, not whichever happened to be last in the array. Motion is **off by default**: a label sliding around a circle is hard to read, so the orbit sits still until asked to move.

The orbit is deliberately not the only thing on the page. A picture cannot tell you which three tasks are late or how the hours split, so a statistics panel and a full table carry that.

The bottom of the page lists projects, not tasks

It listed every task at first, which made it a second copy of the same information twice over — once as planets, once as rows. A department's work is not one flat pile anyway: it is several projects being pushed along at once, and "eleven open in Engineering" hides which of them is in trouble. One row per project answers that directly, and the tasks inside are one click away on the project page.

- Each row carries the project's **whole-company** progress, not just the slice in this department — half a project is not an answer to "how is the portal going".
- Health badge, task counts, hours left, and **assigned and due dates side by side**, so it is possible to see how long something has been running as well as when it is wanted.
- Sorted by trouble: late first, then needing owners, then by how much of this department's work sits in it.
- Clicking a planet still goes straight to that task. The orbit is unchanged.

Single tasks have their own section

Work that belongs to no project is listed separately underneath rather than mixed in or hidden. A one-off assignment with no project around it is the easiest thing in any tracker to lose sight of, so it keeps its own record — with the same owner, stage, assigned date, due date and progress columns as anything else.

Shared pieces live in shared files

Three screens show a project's health badge and two compute days-until-a-date, so both live in `lib/format.js` rather than being copied into each page. The same was done for the CSS: the segmented filter, the health colours, the error banner and the small avatar started inside whichever page needed them first and now sit in `styles/theme.css`. A page should never have to import another page's stylesheet to look right, and a project reading "Late" on one screen and something else on another would be its own small bug.

7.3 State and data loading

Local component state with hooks - no Redux, no state library. The application is small enough that a store would be indirection without benefit. Derived values (statistics, the sorted list) use `useMemo` so they are not recomputed on every keystroke.

7.4 Responsive design

Hand-written CSS with custom properties for the palette in `styles/theme.css`. Media queries in seven stylesheets collapse the multi-column layouts to a single column on narrow screens. The wide task table is wrapped in a container with `overflow-x: auto`, so it scrolls inside itself rather than forcing the whole page sideways - verified in the browser at mobile width, with no horizontal page overflow.

8. What was tested, and the result

The feature was exercised end to end against the live Atlas database, not simulated.

Test	Result
Upload clinic_brief.pdf to POST <code>/api/ai/analyse</code>	HTTP 200. 12 tasks, correct titles, spread across all four departments, 9 with a suggested owner.
Same document as .docx	Identical 12 tasks.
The original project_brief.pdf (regression)	Still 10 tasks, routing improved.
POST <code>/api/ai/create-tasks</code>	HTTP 201, {created: 3}. Task count went 27 -> 30. Records carried source 'ai' and the aiReason text.
Tick subtask 1 of 2	progress 50%, status auto-moved todo -> in_progress.
Tick subtask 2 of 2	progress 100%, status auto-moved to review (not done).
Post a question	HTTP 201, stored with kind 'question'.
Upload then download an attachment	4530 bytes in, 4530 bytes out, content type application/pdf. Byte-exact round trip.
Create a task with an assignee and objective from the UI form	Saved correctly, after the fix in section 9.
Unsupported file type (.xyz)	HTTP 400, readable message naming the supported types.
Oversized file (9 MB)	HTTP 413, readable message.
Bad ObjectId in the URL	HTTP 400, "That record id is not valid."
Missing task	HTTP 404.
Frontend production build	Builds clean, 40 modules, no errors.
Mobile width layout	No horizontal page overflow; table scrolls in its own container.

Test	Result
Click the drop area to open the file picker	One picker request per click (was two, which the browser cancelled). File chosen, analysed, 12 tasks returned.
Department chip vs sidebar 'late' count	Both read 1 for Engineering, and agree with the red halo on the orbit. Previously 2 and 1.

All test records created during this verification were deleted afterwards; the database was returned to its seeded state of 27 tasks.

9. Bugs found during testing, and how they were fixed

These were all found by actually running the feature. Being able to talk about them is worth more than pretending the first version was perfect.

9.1 Assigning a person or an objective silently failed (most serious)

Symptom. Creating a task from the single-task form with an owner or an objective selected returned HTTP 400: *Cast to ObjectId failed for value "Ayan Mahmud - Backend Engineer"*.

Cause. The options endpoint returns `Mongoose .lean()` documents. A lean document is a plain object and carries no virtual `id` field - only `_id`. The JSX read `person.id`, which was undefined, so React rendered `<option>` with no value attribute at all. When an option has no value, the browser falls back to using the option's own *text* as the value - so the form submitted a person's name where an `ObjectId` was expected.

Fix. Use `_id` in all three places (assignee dropdown, objective dropdown, and the preview lookup), matching what the document-import screen already did correctly. The same bug was present on the task detail page's owner dropdown and was fixed there too. The missing React key warning in the console was the visible symptom of the same root cause.

9.2 The file picker would not open in the document import (reported in use)

Symptom. Clicking the drop area on *Add work -> From a document* did nothing. No file dialog appeared, so a PDF or Word file could not be chosen at all. Drag-and-drop still worked, which is what made it look intermittent.

Cause. The hidden `<input type="file">` was a *child* of the drop area, and the drop area's `onClick` called `fileInput.current.click()`. That programmatic click dispatched a click event on the input, which then bubbled straight back up to the drop area, whose handler asked for the picker a second time inside the same user gesture. A browser treats a repeated file-picker request within one gesture as a duplicate and cancels it, so the dialog never appeared. Instrumenting the page confirmed it: one real click produced two calls to the input's `click()`.

Fix. The input was moved out of the drop area so it is a sibling rather than a child. There is now no bubbling path back into the handler, which fixes the cause rather than suppressing the symptom - and it matches how the attachment upload on the task detail page was already written. Two smaller repairs went in alongside: the input's value is cleared after each selection, so picking the same file again after pressing Remove still fires a change event; and the drop area, which carries `role="button"`, now responds to the space bar as well as Enter.

Verified after the fix: one user click produces exactly one picker request, and uploading `clinic_brief.pdf` through the interface returns the expected 12 tasks, 96 hours, 9 with an owner.

9.3 A bodyless POST crashed with a raw TypeError

Symptom. `POST /api/ai/analyse` with no body returned *Cannot read properties of undefined (reading 'text')* - an internal JavaScript error leaking to the user.

Cause. Express 5 leaves `req.body` as undefined when no parser matched the request, rather than defaulting it to an empty object as Express 4 did.

Fix. One small middleware in `server.js` defaults `req.body` to `{}`. Every handler now returns its own validation message instead. Four endpoints were affected.

9.4 Unclassifiable tasks were dumped on the smallest team

Symptom. Tasks the reader could not classify - "Deploy the system to a staging environment" - all landed in **Design**, a two-person team, while Engineering had six people.

Cause. The fallback was `departments[0]`, and the departments query sorts by name. "Design" simply sorts before "Engineering". The routing was an alphabetical accident, not a judgement.

Fix. A default `Department()` helper picks the department with the most active people, ties keeping their original order. Used by both the built-in reader and the Gemini path (where it also catches a department name the model invented).

9.5 Em dashes vanished from generated demo PDFs

Symptom. "Client Brief - Shefa Clinic" lost its dash entirely when the text was read back out.

Cause. The demo PDF generator wrote its text buffer as `latin1`. An em dash (U+2014) is not representable in `latin1`, so it was destroyed on the way in. Apostrophes had a second problem: with no encoding declared, a PDF Type1 font uses `Adobe StandardEncoding`, where byte 39 is a curly quote rather than a straight apostrophe.

Fix. Declare the font `/WinAnsiEncoding` and map the common typographic characters to their `WinAnsi` byte values before writing.

9.6 Section headings were invisible to the reader in PDFs

Symptom. Every drafted task carried the generic reason "Taken directly from a line in the document" instead of naming its section.

Cause. `isHeading()` recognised Markdown hashes, trailing colons and all-caps. A PDF keeps none of those. Section titles were indistinguishable from body text.

Fix. Recognise a heading by shape as well - a short capitalised line, few words, no terminal punctuation, not starting with an action verb. Reasons now read "Listed under 'What we need built' in the document", and descriptions carry their section as context.

9.7 A word-matching false positive sent marketing work to Engineering

Symptom. "Prepare the launch campaign for the clinic's Facebook page" was routed to Engineering.

Cause. The word `page` was in the frontend skill vocabulary, and it matched "Facebook page". **Fix.** Removed `page` - it reads as frontend work about as often as it reads as a social page or a page of a document - and added genuine marketing vocabulary (announcement, newsletter, press, social). That task now routes to Marketing.

9.8 The server only found its .env if started from the right folder

Cause. `dotenv.config()` resolves against the current working directory, so `node backend/server.js` from the project root silently loaded no configuration and failed with a missing connection string. **Fix.** Anchor the path to `__dirname` in both `server.js` and `seed.js`.

9.9 The department chip and the sidebar disagreed about what 'late' means

Symptom. The Engineering chip at the top of the orbit page read **2 late** while the statistics panel on the right read **1 late**, on the same screen at the same moment.

Cause. Two separate definitions of overdue. The per-task field used `Math.ceil((due - now) / oneDay) < 0`, which treats the whole of the due day as not-yet-late. The department count used a raw `dueDate < new Date()`, which calls a task late from one minute past midnight on the day it is due. A task due today was therefore counted by one and not the other - and the task list was simultaneously labelling it "due today".

Fix. A single `isOverdue()` helper in the service layer, used by both. A task is late only once its day has fully passed, which is what "3 days late" in the list already meant. Both now read 1, and they agree with the red halo drawn on the orbit.

This is the same class of bug as the progress field

Two places computing the same fact will eventually disagree. The cure is the same in both cases: one definition, called from everywhere that needs it.

9.10 A guard for an unseeded database

If no departments exist, both planners would have returned tasks with no department, and the failure would only have surfaced later as a validation error on save. `/api/ai/analyse` now returns a clear 409 telling the user to run the seed script.

10. Running the project

10.1 Configuration

Copy backend/.env.example to backend/.env and fill it in:

```
MONGODB_URI=mongodb+srv://<user>:<password>@<cluster>/employee_management?appName=Cluster0

# Optional. Without it the document import falls back to the built-in reader.
# Free key from https://aistudio.google.com/apikey
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.5-flash

PORT=4000
```

The real .env is listed in .gitignore and must never be committed.

10.2 Commands

```
# Backend
cd backend
npm install
npm run seed      # load the demo data (safe to re-run)
npm run dev       # nodemon on port 4000

# Frontend, in a second terminal
cd frontend
npm install
npm run dev       # Vite on port 5173, proxying /api to port 4000
```

10.3 Demonstrating the feature

Open <http://localhost:5173>, go to **New task**, choose **From a document**, and drop in demo/clinic_brief.pdf. Expect 12 tasks, 96 hours estimated, 9 with an owner, spread across all four departments. demo/README.md lists the exact expected output so you can tell at a glance whether something is wrong.

Regenerate the demo files after editing the Markdown source:

```
node demo/make_demo_files.js clinic_brief
```

11. Questions to be ready for

Question	Answer
Why MongoDB rather than SQL?	The task record is a document with variable-shaped nested data - a checklist of subtasks, a list of attachment ids - and it is always read as a whole. That maps directly onto a document store. In SQL the same thing needs a subtasks table and a join on every read.
Is the AI making decisions on its own?	No. It proposes; a person decides. POST /api/ai/analyse writes nothing to the database. Tasks exist only after the manager reviews the draft and posts to a second endpoint.
What if the Gemini API is down, or the key expires?	The request falls through to the built-in reader, and the response says which engine answered. A demo never dies because of a network blip.
Why is progress not a field?	Because it would disagree with the checklist the moment somebody ticked a box. It is derived on read from the subtasks.
Why does finishing the checklist move a task to 'review' and not 'done'?	Completing the steps means the work is ready to be checked. A person still decides when it is actually finished.
How do you assign to a whole room rather than a person?	Every task must have a department, but assignee is nullable. A task with a department and no assignee is exactly that - work owned by the room, showing as 'no owner' in the statistics.

Question	Answer
Where are uploaded files stored, and why?	As binary inside MongoDB. Vercel's filesystem is read-only, so writing to a folder would work locally and fail in production.
How do you stop an uploaded file being used for an attack?	Downloads are forced with Content-Disposition: attachment, so a stored HTML or SVG file can never render as a page on the app's own origin.
How is the Gemini response parsed reliably?	It is not parsed loosely at all. Gemini is given a JSON response schema and constrained to it, so the reply is either valid JSON of the expected shape or an error.
What happens with a scanned PDF?	It contains no text layer, so extraction returns nothing and the user is told plainly that it needs OCR first, rather than being shown an empty result.
What is the biggest weakness?	The built-in reader judges each line in isolation, so it misses priorities stated in a separate Constraints section. With a Gemini key that limitation disappears.

12. What is not done

Stated plainly, because being asked about a gap you have already identified is much better than being caught by one.

- **Version control.** The folder is not a git repository. The guidelines require GitHub/GitLab/Bitbucket with individual commit history, and warn that copy-pasted code without commits will be penalised. This needs doing before submission.
- **No authentication.** There is no login, and the current user is hardcoded in the header as a placeholder. Comment authors are submitted by name rather than taken from a session. This belongs to another module, but it does mean every endpoint is currently open.
- **No automated test suite.** Verification for this document was done by driving the real API and the real interface. There is no npm test.
- **Subtasks cannot be deleted or reordered** - only added and ticked.
- **Attachments are capped at 5 MB** by the 16 MB MongoDB document limit. A production system would use object storage.
- **No pagination.** The board loads a whole department's tasks at once. Fine at this scale, not at ten thousand.
- **The Gemini key is not set** in the current .env, so the module runs on the built-in reader until one is added. It is free and takes about two minutes.

12.1 Suggested next steps, in order

- Initialise git and have each member commit their own work.
- Add the free Gemini key so the stronger engine is what gets demonstrated.
- Wire the real signed-in user into comment authorship once authentication lands.
- Add a small test suite around the two planners - they are pure functions and the easiest thing in the codebase to test.

File map, for finding things quickly

backend/server.js (app setup, error handler) | backend/routes/ (URL to handler) | backend/controller/ (HTTP, validation) | backend/service/task_service.js (all database access) | backend/service/document_parser.js (file to text) | backend/service/gemini.js (the external API) | backend/service/fallback_planner.js (the built-in reader) | backend/model/ (Mongoose schemas) | frontend/src/lib/api.js (every request) | frontend/src/pages/ (the three screens) | demo/ (sample briefs and the generator)